Задание: «Моделирование архитектуры фон Неймана на языке программирования Си» 2.0

Цель: понять ключевые принципы архитектуры фон Неймана — единое хранилище для данных и команд, последовательное выполнение, наличие счётчика команд, АЛУ и регистров — через написание виртуальной машины на языке Си с использованием структур, указателей и чтения программы из текстового файла.

Общие требования:

1. Объявите массив типа int cpu_state[8], который будет хранить состояние процессора. Ячейки массива имеют следующее назначение:
   - cpu_state[0] — счётчик команд (pc)
   - cpu_state[1] — аккумулятор (acc), хранит текущий результат
   - cpu_state[2] — регистр команд (ir), хранит текущий опкод
   - cpu_state[3] — вектор прерываний (адрес обработчика ошибок)
   - cpu_state[4] — флаг остановки (0 = работает, 1 = остановлен)
   - cpu_state[5] — код последней ошибки (0 = нет ошибки)
   - cpu_state[6] — резерв для будущих расширений
   - cpu_state[7] — резерв для будущих расширений
2. Объявите массив типа `int memory[256]`, который будет моделировать память (в ней хранятся и команды, и данные).
3. Ячейка `memory[0]` зарезервирована под вектор прерываний — адрес, куда передаётся управление при ошибке.
4. Программа читается из текстового файла *program.txt* (числа разделены пробелами или переводами строк, каждое число — одна ячейка памяти).
5. Программа должна работать в цикле **while**, пока не встретит команду 0 (это будет «стоп») или не произойдёт прерывание.
6. Выводите на экран каждую выполняемую команду, её операнд и значение регистра `cpu_state[1]` (аккумулятора) после её выполнения.
7. Используйте только конструкции, которые вы уже изучили.

Уровень «Удовлетворительно»

Варианты программ (выберите один):

0,5	1 — загрузить в аккумулятор значение из ячейки памяти по адресу (прямая адресация) 2 — прибавить к аккумулятору значение из ячейки памяти 3 — вычесть из аккумулятора значение из ячейки памяти 0 — стоп	После кода команды (1, 2 или 3) в массиве сразу идёт адрес ячейки-операнда. При выходе за границы памяти или делении на ноль — прерывание (переход на memory[0]).
1,6	1 — загрузить в аккумулятор значение из ячейки памяти 2 — умножить аккумулятор на значение из ячейки памяти 3 — разделить аккумулятор на значение из ячейки памяти 0 — стоп	После кода команды в массиве сразу идёт адрес ячейки-операнда. При делении на ноль — прерывание.
2,7	1 — загрузить в аккумулятор значение из ячейки памяти 2 — прибавить к аккумулятору значение из ячейки памяти 3 — вывести текущее значение аккумулятора на экран (без изменения) 0 — стоп	Команда 3 не требует операнда. Команды 1 и 2 требуют адрес ячейки.
3,8	1 — увеличить значение аккумулятора на 1 2 — уменьшить значение аккумулятора на 1 3 — прибавить к аккумулятору значение из ячейки памяти 0 — стоп	Команды 1 и 2 не требуют операнда. Команда 3 требует адрес ячейки (следующая ячейка).
4,9	1 — вывести значение аккумулятора на экран 2 — загрузить в аккумулятор значение из ячейки памяти 3 — умножить аккумулятор на значение из ячейки памяти 0 — стоп	Команда 1 просто печатает текущее значение. Команда 2 требует адрес ячейки. После кода команды 3 в массиве сразу идёт адрес ячейки-операнда.

Пример (для варианта 1):

Пример (для варианта 1, файл *program.txt*):

Выполнение:

```
10
0 5 3 0 0
1 1
2 2
5 4
0
```

Содержимое памяти после загрузки:

```
memory[0]=10   ; вектор прерываний
memory[1]=0, memory[2]=5, memory[3]=3, memory[4]=0, memory[5]=0
memory[6]=1, memory[7]=1   ; LOAD memory[1]
memory[8]=2, memory[9]=2   ; ADD memory[2]
memory[10]=5, memory[11]=4 ; DIV memory[4]
memory[12]=0               ; HALT
```

Инициализация cpu_state:

```
cpu_state[0]=6   ; pc начинается с 6 (первая команда)
cpu_state[1]=0   ; acc = 0
cpu_state[2]=0   ; ir = 0
cpu_state[3]=10  ; вектор прерываний = 10
cpu_state[4]=0   ; не остановлен
cpu_state[5]=0   ; нет ошибки
```

Вывод программы:

```
pc=6:  команда 1, operand=1, acc=0
pc=8:  команда 2, operand=2, acc=5
pc=10: команда 5, operand=4, ОШИБКА: деление на 0
pc=10: команда 5, operand=4, ОШИБКА: деление на 0 (повторное
прерывание)
Программа завершена с ошибкой.
.
```

Уровень «хорошо»

Требования уровня 3 + следующее:

Добавьте в вашу систему команду 4 — безусловный переход (JUMP).

- Команда 4 означает: «перейти на указанную ячейку памяти».
- После команды 4 в массиве сразу идёт адрес перехода (целое число — индекс в массиве памяти).
- После выполнения команды 4 значение cpu_state[0] (счётчик команд) заменяется на этот адрес (а не увеличивается как обычно).

Важно:

- Убедитесь, что переход не уходит за пределы массива (иначе — прерывание).
- После команды 4 не увеличивайте cpu_state[0] на 2, а присвойте ему значение из следующей ячейки.

Пример (для варианта 1, файл program.txt):

```
20
1 2 3 4 5
0 5 1
1 6
1 7
2 8
5 9
4 12
0
```

Выполнение:

```
pc=9: команда 1 → адрес 6 → acc = memory[6] = 0 → pc += 2
pc=11: команда 1 → адрес 7 → acc = memory[7] = 5 → pc += 2
pc=13: команда 2 → адрес 8 → acc += memory[8] = 1 → pc += 2
pc=15: команда 5 → адрес 9 → acc /= memory[9] = 0 → прерывание (или переход по условию)
pc=17: команда 4 → адрес 12 → pc = 12
pc=12: команда 0 → стоп
```

Команды между 17 и 12 пропускаются за счёт JUMP.

Уровень «отлично»

Требования уровня «хорошо» + следующее:

1. Добавьте команду 5 — условный переход (JUMP IF ZERO).

• Команда 5 означает: «если значение cpu_state[1] (аккумулятора) == 0, перейти на указанный адрес; иначе — продолжить как обычно».

• После команды 5 в массиве идёт адрес перехода.

• Если значение аккумулятора != 0, то команда игнорируется, и значение cpu_state[0] увеличивается на 2.

Пример:
Массив: {1, 0, 4, 8, 1, 99, 0, 0, 1, 42, 0}
Выполнение:

- значение счётчика команд =0: команда 1 →в массив, который хранит вашу программ= 0, значение счётчика команд = 2
- значение счётчика команд =2: команда 4 → в массив который хранит вашу программ== 0 → перейти на program[3] = 8 → значение счётчика команд = 8
- значение счётчика команд =8: команда 1 → в массив, который хранит вашу программ= 42, значение счётчика команд = 10
- значение счётчика команд =10: команда 0 → стоп
  Команды 1, 99, 0, 0 пропущены.

Требования к оформлению отчёта

Отчёт оформляется в электронной форме и должен содержать следующие разделы:

1. Титульный лист

- По шаблону СибГУТИ
- Название работы: *«Моделирование архитектуры фон Неймана на языке С»*
- ФИО студента, группа
- Вариант задания (указывается номер: 0–9)

2. Цель работы

Кратко (2–3 предложения):

Пример: *Понять ключевые принципы архитектуры фон Неймана — единое хранилище для данных и команд, последовательное выполнение, наличие счётчика команд — через написание простой программы на языке С.*

3. Описание программы

- Указать, какой уровень реализован: «3», «4» или «5».
- Описать логику работы программы:
- Какие команды поддерживает ваша «виртуальная машина»?
- Как организован массив-«память»?
- Как работает счётчик команд?
- Какие особенности реализованы (например, безусловный или условный переход)?
- Привести пример массива программы и пошагово объяснить её выполнение (аналогично примеру из задания).

4. Листинг программы

- Полный исходный код на языке С.
- Код должен быть читаемым: с отступами, комментариями и осмысленными именами переменных.
- Обязательно включить комментарий, поясняющий, как программа отражает принципы архитектуры фон Неймана.

5. Результаты выполнения

- Привести реальный вывод программы (скриншот или текст).
- Указать, какие входные данные использовались (если есть ввод).
- Для уровней «4» и «5» — показать, как работает **JUMP** или условный переход (например, цикл или ветвление).

6. Вывод

Краткое заключение (3–4 предложения):

- Что удалось реализовать?
- Какие принципы архитектуры фон Неймана были продемонстрированы?
- Были ли трудности? Как они были решены?

Важно:

- Отчёт должен быть индивидуальным. Использование шаблонных фраз или копирование чужого кода без понимания не допускается.